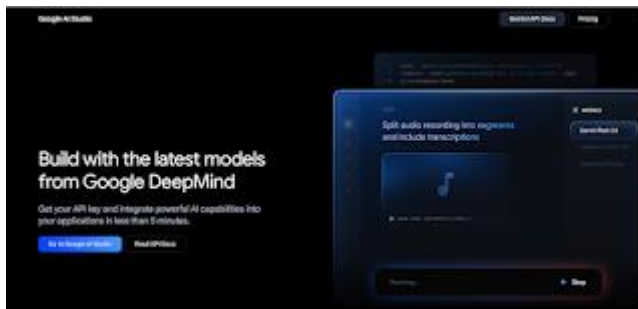


MILESTONE 1: Gemini AI Initialization

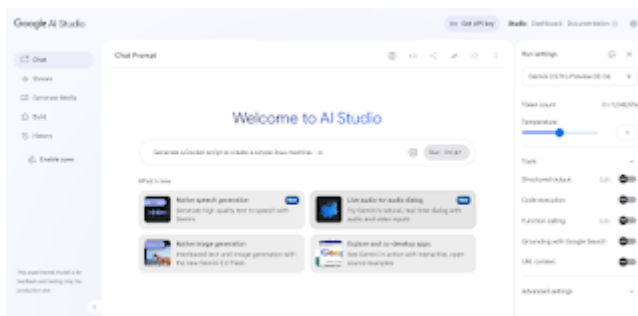
In this foundational stage, the objective is to establish and configure the core AI infrastructure required to power **PocketSmart AI**. The platform relies on **Gemini 1.5 Flash Pro**—Google's multimodal foundation model—to interpret user inputs, process budgets, and generate personalized recommendations. This milestone ensures that the development environment is equipped with authenticated access to Gemini's APIs and is capable of performing both text-based reasoning and image analysis (specifically for the Jewelry Planner). Proper completion of this setup is essential, as it enables all AI-driven functionalities like budget understanding, category-wise recommendations, and multimodal analysis to operate seamlessly.

- Activity 1.1: Set up Google Cloud account and enable Gemini API

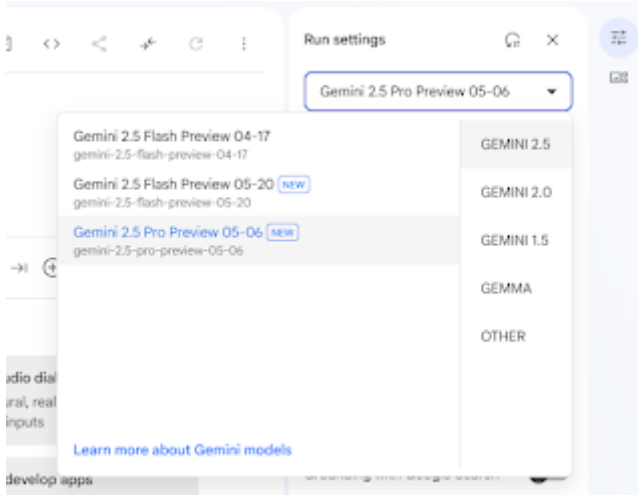
- **Sign in or create a Google Cloud account via console.cloud.google.com.**



- **Accept all terms and conditions and click on Get API key**



- You will get trending models here you can select the model here or you can set the model by go to the API Documentation.

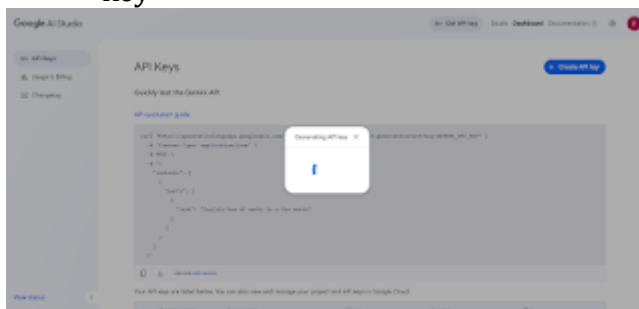


Activity 1.2: Configure access and permissions

- **You will get interface like this and now click on Create API key**



- Once click on API key you will the api key and make sure to copy the API key



- **Activity 1.3: Validate Gemini API connectivity**
 - Make a sample prompt call using Python to confirm integration.

```
import google.generativeai as genai
```

```
from google import genai
```

```
from PIL import Image
```

```
import io
```

```
import base64
```

```
client = genai.Client(api_key="AlzaSyD4h1z1i1i1i1i1i1i1i1i1i1i1i1i1")
```

```
prompt = ("Generate a detailed 3D architectural floor plan for a modern two-story house "
```

```
        "with a total area of 180 meters in length and 70 meters.")
```

```
response = client.models.generate_content(
```

```
    model="gemini-1.5-flash-image-generator",
```

```

        contents=prompt,

        generation_config={"response_mime_type": "image/png"},

    )

```

```

for part in response.candidates[0].content.parts:

```

```

    if part.mime_type == "image/png":

```

```

        img = Image.open(io.BytesIO(part.inline_data.data))

```

```

        img.save("generated-house-image.png")

```

```

        img.show()

```

- Test both text-only and image + text prompts to ensure multimodal compatibility.

```

import google.generativeai as genai
from PIL import Image
import io
import base64

```

```

client = genai.Client(api_key="AlzaSyD4h1z1i1i1i1i1i1i1i1i1i1i1i1i1")

```

```

prompt = ("Generate a detailed 2D architectural floor plan for a modern two-story house "
          "with a total area of 7514 mm in length and 7514 mm in width.")

```

```

response = client.models.generate_content(
    model="gemini-1.5-flash-image-generator",
    contents=prompt,
    generation_config={"response_mime_type": "image/png"},
)

```

```

for part in response.candidates[0].content.parts:

```

```

    if part.mime_type == "image/png":

```

```

        img = Image.open(io.BytesIO(part.inline_data.data))

```

```

        img.save("layout-house.png")

```

```

        img.show()

```